Machine-Learning-Assisted Adaptive Chunk Scheduling for Multiprocessing Workloads

Operating Systems Review 2 Methodology and Partial Implementation

Sashwath J 24BAI0059

This prototype sends independent CPU tasks to a Linux process pool. Fixed chunks of four tasks are the reference. Two alternatives are implemented: a decision tree changes chunk size using live measurements, while a cost-balanced dispatcher groups tasks using known metadata. The results show different strengths and costs. The combined method proposed for the next review is not yet implemented.

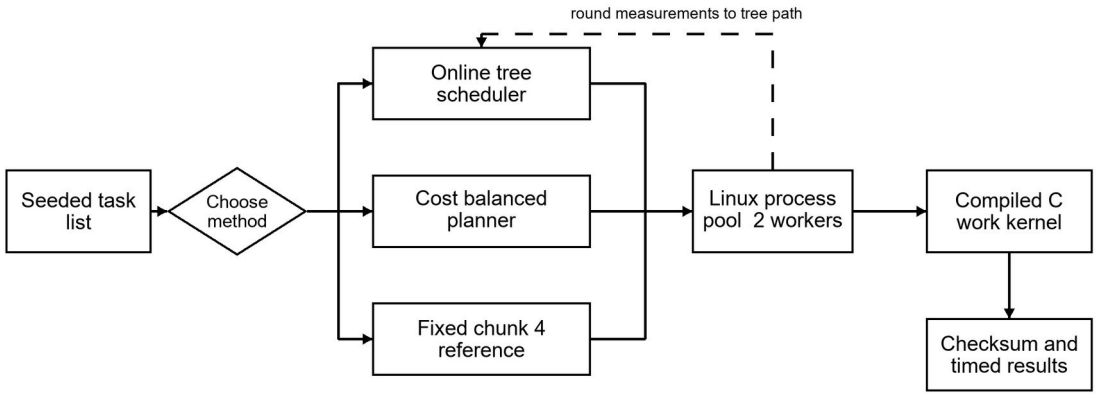
1 Problem and proposed method

A fixed chunk size gives each worker the same number of tasks per batch even when task cost varies. Small chunks can create dispatch overhead; large chunks can leave one worker with a long final batch. The project tests two responses to this problem. The decision-tree path observes early execution and predicts one of four chunk sizes. The cost-balanced path uses available task metadata to make batches of roughly similar estimated cost, then sends longer batches first. The Linux kernel still schedules the worker processes; this project changes user-space task dispatch, not kernel code.

The next question is whether a combined scheduler can use the tree's workload signal and cost balancing without the current tree path's repeated round barrier. This is a testable design, not a measured result. Machine-learning-assisted task scheduling and adaptive self-scheduling already exist in the literature (Qawasmeh et al., 2016; Booth & Lane, 2022; Nookala et al., 2024).

2 System architecture

Figure 1 Implemented user space scheduling architecture. The dashed line shows runtime feedback used only by the tree path.



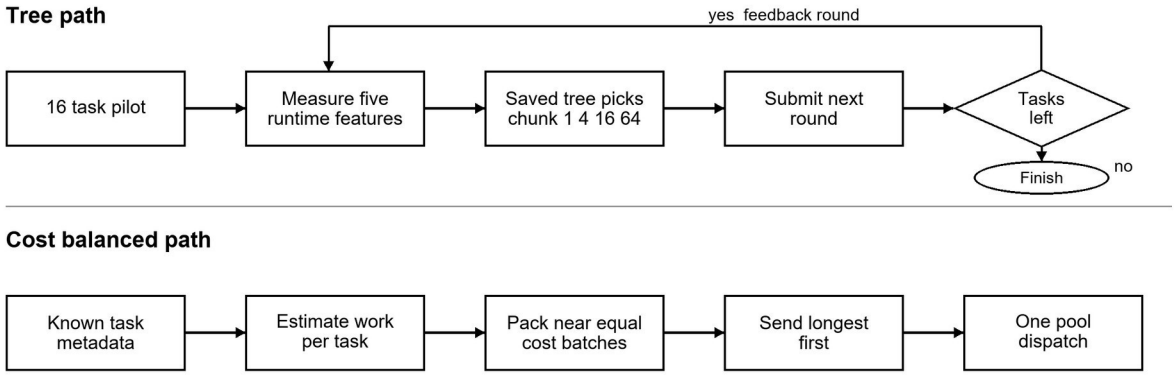
The controller runs in Python 3.12.3 on Ubuntu under WSL2. It creates tasks, chooses fixed, tree or costbalanced dispatch, and sends batches to two worker processes. Each worker calls the compiled C11 CPU kernel through ctypes. The parent records timing and checksums after results return. Model fitting is offline; the measured run loads an already trained tree.

3 Major modules and workflows

Module	Implemented responsibility	Main output

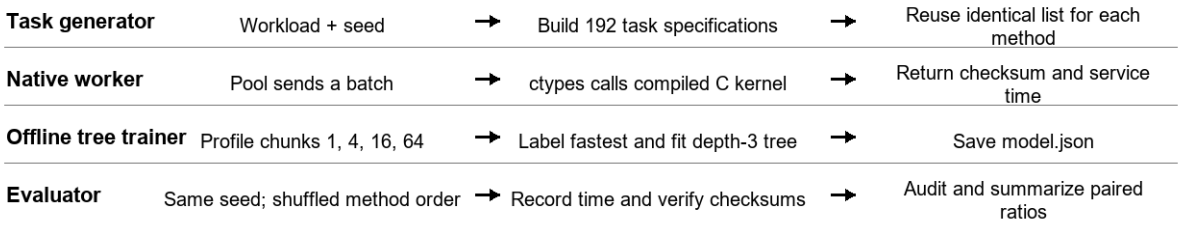
Task generator	Recreates 192 CPU tasks from a workload name and seed.	Task list and seed
Native worker	Executes synthetic iteration work or prime counting in C.	Checksum and service time
Tree trainer	Profiles candidate chunks, labels fastest choices, fits a depth-3 tree.	model.json
Module	**Implemented responsibility**	**Main output**
Tree scheduler	Runs a pilot, measures feedback, predicts the next chunk per round.	Decision trace and timing
Cost balancer	Estimates task cost, packs batches, sorts them longest first.	Batches and timing
Evaluator	Times matched runs and audits checksums and summary medians.	raw.csv and summary.csv

Figure 2 The two implemented scheduling paths. The tree waits at feedback rounds; cost balancing prepares batches once.



The generator first creates the same seeded task list for each method. Offline training profiles candidate chunks and saves model.json; the evaluator later times each method on matched inputs, checks output equality, and recomputes the paired statistics. These training and evaluation steps run outside the two dispatch paths shown in Figure 2.

Figure 2a Workflows for the remaining implemented modules. Training is offline; evaluation uses matched tasks and checksums.



4 Dataset and collection procedure

The inputs are reproducible generated CPU workloads, not an external dataset or collected production trace. Every timing group uses the same task list for all three methods. Five general patterns each have five seeds (7001-7005); the late-heavy pattern has ten seeds (11001-11010). Each group contains 192 tasks on two workers. Training uses different seeds (11-16), the holdout uses 101-104, and the paired benchmark uses the later seeds. This separation limits direct train/test leakage.

Pattern	How the 192 tasks are generated	Why included
Uniform	Each synthetic task has 1.5 million C iterations.	Low variability control
Moderate	Each task scales 1.5 million iterations by a seeded factor from 0.5 to 1.5.	Ordinary uneven work
High	70 percent are about 0.2 times and the remainder about 2.87 times base work.	Strong size variation

Phase shift	First half is light; second half mixes very light and heavy work.	Distribution change during a run
Prime scan	C counts primes in successive 3,000-integer ranges.	Non-synthetic computational kernel
Late heavy	One of the last eight tasks has 40 times base iterations; others vary 0.9-1.1 times.	Rare delayed straggler

The training table has 90 fitted rows and 60 held-out rows. Each row is a measurement at a liveequivalent decision boundary. Its five predictors are mean task service time (average worker C time per task), batch spread ((maximum minus minimum batch service time) divided by maximum), throughput (tasks completed divided by round seconds), current chunk size, and fraction of tasks remaining. The label is the fastest median candidate among chunks 1, 4, 16 and 64; choices within 3 percent of fastest prefer the larger chunk. These sizes span one task through coarse batching, rather than being arbitrary output classes. The held-out exact label accuracy is 73.3 percent (44 of 60); this is a classification result, not a runtime speedup.

Collection is deterministic from the generator and saved seeds. The earlier full run saved 4,800 task records in results_final_linux/dataset.csv. The paired study stores each group seed, method order, full timings and checksum in raw.csv; its generator code recreates the exact task list. No personally identifiable or third-party data is used.

5 Tools technologies and algorithms

Execution used Ubuntu on WSL2, Python 3.12.3 for the controller and multiprocessing pool, GCC 13.3 for the C11 work kernel, ctypes for Python-to-C calls, and the Python standard library for a custom CART-style decision tree and CSV/JSON output. The fixed reference groups four tasks. The tree is depth 3 with minimum leaf size 5. The cost method sets target cost to total estimated cost divided by 32, greedily packs adjacent tasks up to 32 items, sorts batches by estimated cost and dispatches them once. Synthetic cost is estimated from the known iteration count. Prime cost uses work multiplied by an approximate square-root scale of the number range. The cost estimate is task metadata, not a learned prediction.

6 Experimental plan and measures

The planned comparison is fixed chunk 4 versus online decision tree versus cost-balanced 32 under matched tasks. For each workload and seed, method order is seeded and shuffled. The primary measure is the median across seeds of each method's wall-time ratio to fixed 4 on the same seed. A ratio below 1 means lower full elapsed time; 0.870 means about 13 percent less time. Wall time starts before batch planning or pool creation and stops after results return and pool teardown. Offline model training is excluded from a test run but reported separately. This boundary measures application-level execution, not a kernel scheduling event.

Measure	Definition	Interpretation
Full wall time	Time from scheduling start through planning, pool startup, dispatch and teardown.	Primary user-visible latency; lower is better.
Paired ratio	Method time divided by fixed-4 time for the same input seed; median of ratios.	Below 1 faster; above 1 slower than baseline.
Batch p95	Per-run 95th percentile from batch submission to worker completion.	Shows slow-tail behavior; not the same as total wall time.
Startup	Time from run start until worker pool is ready.	Part of full wall time.
Checksum	Sum of deterministic C results for all 192 tasks.	Equality checks output correctness across methods.

The benchmark has 35 matched input groups and 105 timed method runs. Its saved audit recomputes all summary medians, checks method coverage and order, and confirms equal checksums in each group. The sample is exploratory on one WSL2 machine with two logical CPUs. No confidence interval or crossmachine generalization is claimed.

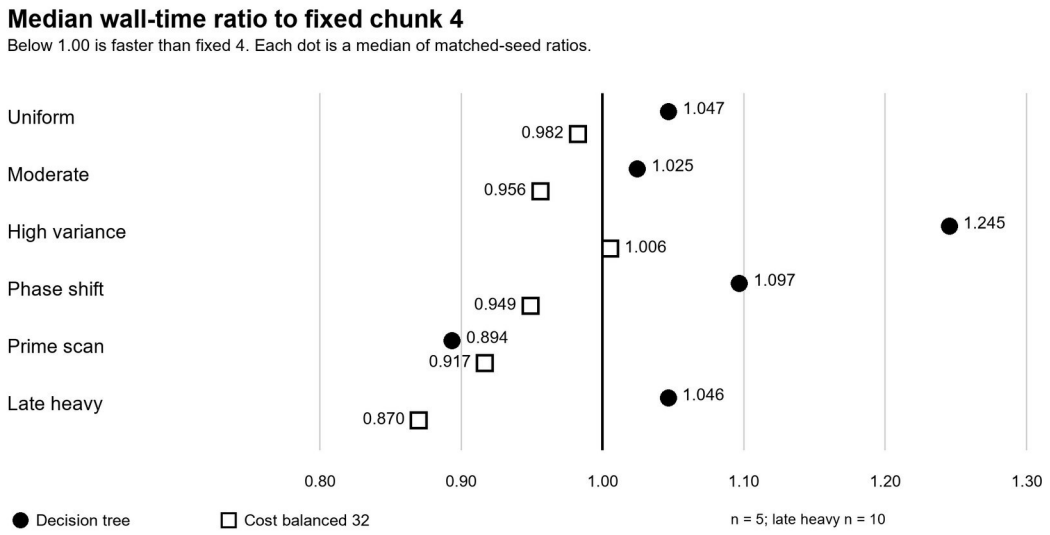
7 Measured comparison and interpretation

Table 1 Full wall time and median paired ratio by workload

Workload	n	Fixed 4 s	Tree s / ratio	Balanced s / ratio
uniform	5	0.365	0.385 / 1.047	0.358 / 0.982
moderate	5	0.403	0.399 / 1.025	0.367 / 0.956
high	5	0.366	0.456 / 1.245	0.364 / 1.006
phase shift	5	0.372	0.411 / 1.097	0.368 / 0.949
prime scan	5	0.235	0.212 / 0.894	0.201 / 0.917
late heavy	10	0.496	0.522 / 1.046	0.432 / 0.870

Values are medians over five seeds per general workload and ten late-heavy seeds. The ratio is the median of within-seed ratios, so it need not equal a ratio of the two displayed median seconds. This matters for moderate and prime scan, where comparing the seconds alone could give a different ranking from the paired statistic.

Figure 3 Paired wall-time ratios calculated directly from the saved benchmark summary. The vertical line is the fixed-4 baseline.



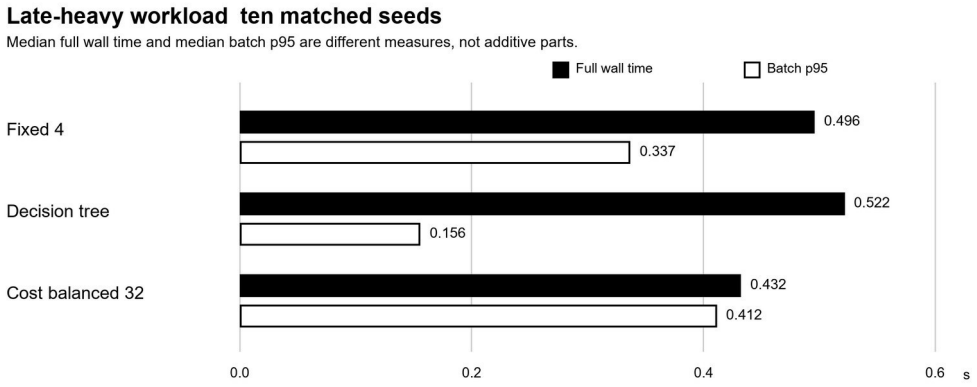
The tree improves the paired ratio on prime scan (0.894) but is slower than fixed 4 on uniform, moderate, high, phase-shift and late-heavy tasks. Its high-variance ratio is 1.245, about 24.5 percent more time. A plausible contributor is the repeated round barrier in the current implementation: each model decision waits for a round's submitted batches to finish. Prediction itself takes only microseconds in the live trace, but the full dispatch pattern can still be slower. The present data do not isolate a single cause.

Cost balancing is strongest on the intended late-heavy pattern: median paired ratio 0.870, or about 13 percent less full time than fixed 4; median wall times are 0.432 s versus 0.496 s. It also improves the paired ratio on uniform, moderate, phase-shift and prime scan, but is slightly slower on high variance (1.006). This is evidence for a targeted benefit, not universal superiority. It does not make the cost method machine learning.

Table 2 Late heavy diagnostics Ten matched seeds

Method	Wall s	Paired ratio	Batch p95 s	Startup s
Fixed 4	0.496	1.000	0.337	0.005
Decision tree	0.522	1.046	0.156	0.005
Cost balanced 32	0.432	0.870	0.412	0.006

Figure 4 Late-heavy full wall time and batch p95 from ten matched seeds. The two measures describe different timing boundaries.



A lower batch p95 does not guarantee lower total wall time. In the late-heavy group the tree has the lowest batch p95 (0.156 s) but slower total time (0.522 s). The cost path starts its estimated heavy batch early; in the live demonstration, task 187 moved to dispatch position 0. Its batch p95 is higher (0.412 s) while total time is lower. The single live run itself is not used as the ten-seed effect estimate; its exact elapsed times can change with system load.

8 Implementation progress and valid outputs

Six of seven planned modules work: task generation, compiled C execution, tree training, online tree scheduling, cost-balanced scheduling, and paired evaluation. The combined overhead-aware scheduler remains for the next stage. Module count is not a precise measure of effort, but the working prototype exceeds the Review 2 half-complete requirement. No hybrid benchmark is claimed.

Delivered module	Evidence to show	Status
Generator and C kernel	workload.py; native/work_kernel.c; repeatable checksums	Complete
Tree training	training_data.csv; model.json; 73.3 percent held-out accuracy	Complete
Online tree run	live demo trace: chunk 4 to 16; equal checksums	Complete
Cost-balanced run	live demo: late task 187 dispatched first; equal checksums	Complete
Paired evaluator	105 raw rows; 18 summary rows; all groups audited	Complete
Automated checks	12 unit tests passing in Ubuntu	Complete
Combined scheduler	Design and ablation pending	Remaining

9 Challenges and remaining work

Chunk-label accuracy did not predict wall time; the tree's round barrier adds waiting, while cost balancing needs useful task metadata. Synthetic tasks and one two-worker WSL2 host limit generalization.

Next, test an overhead-gated combined scheduler: use the tree after a pilot, balance only unstarted tasks when metadata are informative, and remove unnecessary round barriers. Compare fixed 4, tree, cost balance and the combined path on matched seeds. Then perturb cost estimates, test more workers and longer tasks, report uncertainty intervals, and repeat on another Linux machine or real trace. Keep the combined path only if full wall time improves with equal outputs.

Terminal results

The WSL project has a logical directory alias at the prompt shown below. The screenshots are cropped for print readability; the white-on-black text is the original terminal display.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ python3 -m unittest discover -s tests -q
----------------------------------------------------------------------
Ran 12 tests in 0.365s

OK
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A1 Fresh Linux unit-test run: 12 tests, OK.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ python3 -m os_sched.verify_paired
PASS  12-column raw data and 18 summary rows audited
PASS  35 paired input groups / 105 method runs
PASS  all three checksums equal within every group
PASS  summary wall times and paired ratios recomputed

Median paired wall-time ratio to fixed chunk 4 (lower is better)
workload        n     tree  balanced
uniform         5    1.047     0.982
moderate        5    1.025     0.956
high            5    1.245     1.006
phase_shift     5    1.097     0.949
prime_scan      5    0.894     0.917
late_heavy     10    1.046     0.870
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A2 Saved benchmark audit: 35 paired groups, 105 runs, equal checksums and recalculated ratios.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ bash show_both_review2.sh
Tree scheduler (first decisions and final check):
LIVE prime_scan tasks=192 workers=2 model=results_final_linux/model.json
round=01 chunk=4->16 completed=16 decision_us=22.2
round=01 chunk=4->4 completed=16 decision_us=17.7
END fixed4=0.182s tree=0.238s guarded=0.184s checksums_equal=True output=/tmp/review2_tree.json
Cost-balanced scheduler (late-heavy run):
Tail spike at task 187; balanced scheduler moves its batch to dispatch position 0
fixed4=0.479s pool_map16=0.517s balanced32=0.427s checksums_equal=True
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A3 Fresh live runs of both paths: the tree changes chunk 4 to 16; cost balancing dispatches the late task first. Both pass checksum checks. The tree trace is filtered to its first decisions and final result.

References

Booth, J. D., & Lane, P. A. (2022). An adaptive self-scheduling loop scheduler. Concurrency and Computation: Practice and Experience, 34(6), e6750. https://doi.org/10.1002/cpe.6750

Nookala, P., Chard, K., & Raicu, I. (2024). X-OpenMP: eXtreme fine-grained tasking using lock-less work stealing. Future Generation Computer Systems, 159, 444-458. https://doi.org/10.1016/j.future.2024.05.019

Qawasmeh, A., Malik, A. M., & Chapman, B. M. (2016). Adaptive OpenMP task scheduling using runtime APIs and machine learning. In Proceedings of the 2015 IEEE 14th International Conference on Machine Learning and Applications (pp. 889-895). https://doi.org/10.1109/ICMLA.2015.111

Project repository: https://github.com/Sashwath-sash/ml-adaptive-chunk-scheduling